

第八次上机作业

学号：241830137 姓名：朱子健

2026 年 4 月 29 日

1 问题

1. 广义多项式拟合

把 `polyfitEx2.py` (Python 代码) 或 `polyfitEx2.m` (Matlab 代码) 的 TODO 部分改成可以用下面的广义多项式拟合：

$$f(x) = a_0 + a_1 f_1(x) + a_2 f_2(x) + \cdots + a_n f_n(x).$$

注：这里采用的验证算例是用 $1, x, \cos x, \sin x$ 这四个函数的线性组合来拟合前面的数值算例。

提示：对于 Matlab 代码，你可能需要查阅“元胞数组”的使用方法。

2. 多项式拟合与插值

数据

x	0.000	0.895	1.641	2.512	3.542	4.054	4.602	5.063	5.354	5.617
y	1.000	1.803	3.680	7.320	13.591	17.412	22.192	24.892	26.552	29.777

的 9 次多项式拟合可以得到拟合曲线

问题

1. 得到的均方误差和矩阵 $\mathbf{A}$ 的条件数是多少？

提示：在 Python 中可以用 `numpy.linalg.cond` 来输出条件数；在 Matlab 中可以用 `cond` 来输出条件数。

2. 可以证明如果用 9 次多项式拟合，那么当 f 是 Lagrange 插值多项式时，均方误差是 0。但 (1) 中均方误差的结果并不是 0，可以依此认为在这个例子中 9 次多项式插值和 9 次多项式拟合不是一回事吗？
3. 前面多项式空间 P^n 采用的基是 $\{1, x, \dots, x^n\}$ 。把多项式空间 P^n 的基改成 Chebyshev 多项式 $\{T_k(x) : k = 0, 1, \dots, n\}$ ，然后采用“广义多项式拟合”一节的方法来推导矩阵 A 并求出拟合函数，观察均方误差和 A 的条件数会怎么表现。

3. 手动实现梯度下降法

(1) 代码补全

请补全 `nonlinfitEx2.py` (Python 代码) 或 `nonlinfitEx2.m` (Matlab 代码) 的 TODO 部分, 完成梯度下降法优化函数

```
optimize(f, df, x0, eta, eps = 1e-8),
```

输入值分别为函数 f 、它的梯度 df 、初值 x_0 、步长 η 、终止准则的误差容限是 ϵ 。在程序中, 我们首先判断输入参数的维数是否对应, 如果维数不对应就返回一个错误: `Dimension does not match.`

(2) 数据拟合

x	-1.5	-1	-0.5	0	0.5	1	1.5
y	0.04	0.17	0.65	1.39	1.85	1.90	1.99

用形如 $y = \frac{2}{1+ae^{bx}}$ 的函数来逼近上面的数据。先用线性化技巧得到了一个初值, 再用你写的梯度下降法来得到一个更好的值, 并画出拟合函数的图像和损失函数随着迭代步数的变化图像。

4. COVID-19 疫情 Logistic 模型

COVID-19 冠状病毒疫情爆发初期 (2020 年 1 月 16 日至 2020 年 3 月 16 日) 武汉市的确诊人数在附件的 `wuhan2020.csv` 中存储。Logistic 模型

$$\frac{dP}{dt} = r \left(1 - \frac{P}{K} \right) P$$

可以用来刻画时间 t 的感染人口数 P 。其中, K 是最大容量, r 是增长率。若初值为 $P(0) = P_0$, 则这个微分方程的解是

$$P(t) = \frac{K}{1 + \left(\frac{K}{P_0} - 1 \right) e^{-rt}}. \quad (*)$$

把 (*) 式作为拟合函数, 根据给定的确诊数据来给出 P_0, K 和 r 这三个参数的预测值。拐点是满足 $P''(t_0) = 0$ 的点 t_0 。

(1) 全数据预测

根据 1 月 16 日至 3 月 16 日的确诊数据, 给出 P_0, K, r 的预测值, 并确定拐点的日期。

(2) 早期数据预测

假如今天是 2020 年 2 月 1 日, 仅知道 1 月 16 日至 1 月 31 日这段时间的数据, 用这部分数据预测 P_0, K, r 的值是多少? 根据这些预测值得到的模型, 估计 2 月 1 日至 3 月 16 日的确诊人数和拐点日期, 并与真实值作比较。(拐点可与 (1) 中求出的拐点作比较)

注: 调用 csv 文件的方法已在 `nonlinfitEx3.py` 和 `nonlinfitEx3.m` 给出。

2 算法原理与设计

2.1 算法原理

2.1.1 广义多项式拟合

考虑拟合函数 $f(x)$ 线性依赖于参数 $a_1, a_2, \dots, a_n$ 的情形。按照均方误差定义的优化问题称为**线性最小二乘问题**。

设有一组数据 $\{(x_i, y_i)\}_{i=1}^m$ ，其中 $x_i \in \mathbb{R}^d, y_i \in \mathbb{R}$ 。寻求一定类型的函数 $y = f(x)$ 来拟合这些数据，使得在给定点的误差按照某种度量达到最小。常采用均方误差

$$\mathcal{L}(f) = \frac{1}{m} \sum_{i=1}^m (f(x_i) - y_i)^2$$

来衡量给定点的误差。

函数 f 的表达式一般由某些参数 $a_0, a_1, \dots, a_n$ 确定，按照误差最小的原则上述数据拟合问题转化为优化问题：

$$\min_{f \in \mathcal{F}} \mathcal{L}(f),$$

其中 $\mathcal{F}$ 是由一族依赖于参数 $a_0, a_1, \dots, a_n$ 的函数构成的集合，例如不超过 n 次的多项式函数构成的集合 P_n 。若 $\mathcal{L}(f)$ 是均方误差，那么把优化问题的解 f 称为数据 $\{(x_i, y_i)\}$ 的**最小二乘拟合函数**。用不超过 n 次的多项式函数

$$f(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$$

来拟合 m 个数据 $\{(x_i, y_i)\}$ ，即拟合函数空间为 P_n 。

根据极值存在的必要条件，当 $a_0, a_1, \dots, a_n$ 使得

$$\mathcal{L}(a_0, a_1, \dots, a_n) := \mathcal{L}(f) = \frac{1}{m} \sum_{i=1}^m (f(x_i) - y_i)^2$$

取最小值时，有

$$\frac{\partial \mathcal{L}}{\partial a_k} = \frac{2}{m} \sum_{i=1}^m x_i^k (a_0 + a_1 x_i + \dots + a_n x_i^n - y_i) = 0, \quad k = 0, 1, \dots, n.$$

把这 $n+1$ 个方程转化为线性方程组的形式。对向量 $\mathbf{x} = (x_1, x_2, \dots, x_m)^T$ 记向量的幂次为 $\mathbf{x}^k = (x_1^k, x_2^k, \dots, x_m^k)^T \in \mathbb{R}^m$ ，并引入内积记号，可改写为如下线性方程组：

$$A\mathbf{u} = \mathbf{b},$$

其中

$$A = \begin{pmatrix} m & \sum x_i & \dots & \sum x_i^n \\ \sum x_i & \sum x_i^2 & \dots & \sum x_i^{n+1} \\ \vdots & \vdots & \ddots & \vdots \\ \sum x_i^n & \sum x_i^{n+1} & \dots & \sum x_i^{2n} \end{pmatrix}, \quad \mathbf{u} = \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} \sum y_i \\ \sum x_i y_i \\ \vdots \\ \sum x_i^n y_i \end{pmatrix}.$$

只需求解上面的线性方程组即可得到多项式拟合函数。该方程组也称为**法方程组**可以把上述多项式形式拟合中的基函数改为任意的线性无关函数系 $\{\phi_j(x)\}_{j=0}^n$ 。那么对数据 $\{(x_i, y_i)\}_{i=1}^m$ ($n < m$) 的最小二乘拟合问题转化为如下极值问题：

$$\min_{c_0, c_1, \dots, c_n \in \mathbb{R}} \frac{1}{m} \sum_{i=1}^m \left(\sum_{j=0}^n c_j \phi_j(x_i) - y_i \right)^2.$$

记向量 $\phi_k = (\phi_k(x_1), \phi_k(x_2), \dots, \phi_k(x_m))^T \in \mathbb{R}^m$, 引入记号

$$(\phi_j, \phi_k) := \sum_{i=1}^m \phi_j(x_i) \phi_k(x_i), \quad (\phi_k, \mathbf{y}) := \sum_{i=1}^m y_i \phi_k(x_i),$$

则得到相应的法方程组

$$\Phi c = b,$$

其中

$$\Phi = \begin{pmatrix} (\phi_0, \phi_0) & (\phi_0, \phi_1) & \cdots & (\phi_0, \phi_n) \\ (\phi_1, \phi_0) & (\phi_1, \phi_1) & \cdots & (\phi_1, \phi_n) \\ \vdots & \vdots & \ddots & \vdots \\ (\phi_n, \phi_0) & (\phi_n, \phi_1) & \cdots & (\phi_n, \phi_n) \end{pmatrix}, \quad c = \begin{pmatrix} c_0 \\ c_1 \\ \vdots \\ c_n \end{pmatrix}, \quad b = \begin{pmatrix} (\phi_0, \mathbf{y}) \\ (\phi_1, \mathbf{y}) \\ \vdots \\ (\phi_n, \mathbf{y}) \end{pmatrix}.$$

需要指出的是, 这里引入的记号 (ϕ_j, ϕ_k) 并不构成连续空间中的内积。因此, 仅仅凭借 $\phi_0(x), \phi_1(x), \dots, \phi_n(x)$ 的线性无关并不能推出 Φ 是非奇异的。为保证方程组的系数矩阵 Φ 非奇异, 必须加上 Haar 条件: 设 $\phi_0(x), \phi_1(x), \dots, \phi_n(x) \in C[a, b]$ 线性无关。如果 $\phi_0(x), \phi_1(x), \dots, \phi_n(x)$ 的任意 (非零) 线性组合在点集 $\{x_1, x_2, \dots, x_m\}$ 上至多只有 n 个不同的零点, 则 Φ 非奇异。

2.1.2 梯度下降法

一般地, 设 $J(\theta)$ 是 $\mathbb{R}^n$ 中连续可微函数, 要求其最小值及达到最小值时 θ 的取值, 即求解如下的优化问题:

$$\theta^* = \arg \min_{\theta} J(\theta).$$

一个比较自然的寻找方式是: 从某个点 θ_k (初始时 $k=0$) 出发, 找下一个点 θ_{k+1} 使得 $J(\theta_{k+1}) < J(\theta_k)$, 然后不断迭代下去, 直到某个 $J(\theta_k)$ 无法出现显著下降为止。

根据微分中值定理, 假设 θ_k 和 θ_{k+1} 非常接近, 那么

$$\begin{aligned} J(\theta_{k+1}) &= J(\theta_k) + \nabla J(\xi_k) \cdot (\theta_{k+1} - \theta_k), \quad (\xi_k \text{ 在 } \theta_k \text{ 与 } \theta_{k+1} \text{ 之间}) \\ &\approx J(\theta_k) + \nabla J(\theta_k) \cdot (\theta_{k+1} - \theta_k), \end{aligned}$$

其中 ∇J 为 J 的梯度:

$$\nabla J = \left(\frac{\partial J}{\partial \theta_1}, \dots, \frac{\partial J}{\partial \theta_n} \right)^T.$$

这样, 如果我们取

$$\theta_{k+1} - \theta_k = -\alpha_k \nabla J(\theta_k),$$

其中 α_k 是足够小的正数, 那么就能保证

$$J(\theta_{k+1}) \approx J(\theta_k) - \alpha_k |\nabla J(\theta_k)|^2 < J(\theta_k),$$

即由 θ_k 到 θ_{k+1} , $J(\theta)$ 在下降。

于是, 梯度下降法可描述为下面的迭代格式:

$$\theta_{k+1} = \theta_k - \alpha_k \mathbf{g}_k,$$

其中 $\mathbf{g}_k = \mathbf{g}(\theta_k) := \nabla J(\theta_k)$, 非负实数 α_k 称为步长。迭代终止准则可以取为 $|\mathbf{g}_k|$ 小于给定的误差容限。

在梯度下降法的求解过程中，只需求解损失函数的一阶导数，计算代价相比二阶方法（需要用到 Hessian 矩阵的迭代算法）更小，可以在很多大规模数据集上应用。但是，梯度下降法由于方向选择的问题，得到的结果不一定是全局最优，即参数寻优可能陷入局部极小。

2.2 算法设计

算法 1 广义多项式拟合

输入: 数据集 $\{(x_i, y_i)\}_{i=1}^m$ ，基函数系 $\{\varphi_j(x)\}_{j=0}^n$ ，其中 $n < m$

输出: 拟合系数 $c_0, c_1, \dots, c_n$ ，拟合函数 $f(x) = \sum_{j=0}^n c_j \varphi_j(x)$

```

1: 初始化  $(n+1) \times (n+1)$  矩阵  $\Phi \leftarrow \mathbf{0}$ 
2: 初始化  $(n+1)$  维向量  $\mathbf{b} \leftarrow \mathbf{0}$ 
3: for  $j \leftarrow 0$  to  $n$  do
4:   for  $k \leftarrow 0$  to  $n$  do
5:      $\Phi_{jk} \leftarrow \sum_{i=1}^m \varphi_j(x_i) \cdot \varphi_k(x_i)$ 
6:   end for
7:    $b_j \leftarrow \sum_{i=1}^m y_i \cdot \varphi_j(x_i)$ 
8: end for
9: 解方程组  $\Phi \mathbf{c} = \mathbf{b}$ ，得到系数向量  $\mathbf{c} = (c_0, c_1, \dots, c_n)^T$ 
10: 计算均方误差  $\text{MSE} \leftarrow \frac{1}{m} \sum_{i=1}^m \left( \sum_{j=0}^n c_j \varphi_j(x_i) - y_i \right)^2$ 
11:
12: return  $\mathbf{c}$ , MSE

```

算法 2 梯度下降法 (Gradient Descent)

输入: 目标函数 $f(\boldsymbol{\theta})$ ，梯度函数 $\nabla f(\boldsymbol{\theta})$ ，初始点 $\boldsymbol{\theta}_0$ ，步长 $\eta > 0$ ，误差容限 $\varepsilon > 0$ ，最大迭代次数 M

输出: 最优参数 $\boldsymbol{\theta}^*$

```

1:  $k \leftarrow 0$ 
2:  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta}_0$ 
3: while  $k < M$  do
4:    $\mathbf{g} \leftarrow \nabla f(\boldsymbol{\theta})$  {计算当前梯度}
5:   if  $\|\mathbf{g}\|_2 < \varepsilon$  then
6:     break {梯度足够小，收敛}
7:   end if
8:    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \cdot \mathbf{g}$  {沿负梯度方向更新}
9:    $k \leftarrow k + 1$ 
10: end while
11:
12: return  $\boldsymbol{\theta}$ 

```

3 结果分析

3.1 广义多项式拟合——基函数扩展

3.1.1 验证：二次多项式拟合

使用 `polyfit` 函数对给定的 10 个数据点进行二次多项式拟合 ($f(x) = a_0 + a_1x + a_2x^2$), 结果如下:

系数	数值
a_2	0.7542712
a_1	0.98752785
a_0	0.53554452

表 1: 二次多项式拟合系数

拟合函数为:

$$f(x) = 0.7542712x^2 + 0.98752785x + 0.53554452$$

均方误差 $MSE = 0.1751$ 。

3.1.2 广义多项式拟合: $1, x, \cos x, \sin x$

使用基函数系 $\{1, x, \cos x, \sin x\}$ 对相同数据进行广义多项式拟合, 即寻求

$$f(x) = c_0 \cdot 1 + c_1 \cdot x + c_2 \cdot \cos x + c_3 \cdot \sin x$$

使得均方误差最小。

系数	数值
c_3 ($\sin x$ 系数)	-1.90183422
c_2 ($\cos x$ 系数)	2.68270259
c_1 (x 系数)	4.90973928
c_0 (常数项)	-2.15461946

表 2: 广义多项式拟合系数 (基函数: $1, x, \cos x, \sin x$)

拟合函数为:

$$f(x) = -1.9018 \sin x + 2.6827 \cos x + 4.9097x - 2.1546$$

均方误差 $MSE \approx 5.84 \times 10^{-4}$ 。

3.1.3 对比分析

分析 从表中可以看出:

1. 二次多项式拟合的 MSE 为 0.1751, 而使用 $\{1, x, \cos x, \sin x\}$ 的广义多项式拟合的 MSE 降至 5.84×10^{-4} , 降低了约 300 倍。

拟合方法	基函数	MSE
二次多项式拟合	$\{1, x, x^2\}$	0.1751
广义多项式拟合	$\{1, x, \cos x, \sin x\}$	5.84×10^{-4}

表 3: 两种拟合方法对比

2. 这说明在这组数据中, 引入三角函数基 $\cos x$ 和 $\sin x$ 能够显著提高拟合精度。原因可能是数据本身具有某种周期性特征, 单纯用多项式难以捕捉。
3. 该验证算例成功证明了 `polyfit` 函数支持灵活的基函数配置, 用户可以传入任意线性无关的函数系来实现广义多项式拟合。

3.2 广义多项式拟合——基函数对比

我们分别使用单项式基 $\{1, x, \dots, x^9\}$ 和 Chebyshev 多项式基 $\{T_k(x)\}_{k=0}^9$ 对给定的 10 个数据点进行 9 次最小二乘拟合, 结果如下:

基函数	均方误差 (MSE)	矩阵 A 的条件数
单项式 $\{1, x, \dots, x^9\}$	9.58×10^{-7}	3.62×10^{13}
Chebyshev $\{T_k(x)\}_{k=0}^9$	1.32×10^{-18}	1.85×10^3

表 4: 不同基函数的 9 次多项式拟合对比

分析 当使用单项式基时, 法方程组的系数矩阵 A 为 Hilbert 矩阵, 条件数高达 10^{13} , 属于严重病态。尽管数学上当拟合多项式为 Lagrange 插值多项式时 MSE 应为 0, 但由于舍入误差的影响, 实际计算得到的 MSE 并非严格为 0 (约为 9.58×10^{-7})。这说明了在这个例子中, 9 次多项式插值和数值计算出的 9 次多项式拟合确实不是一回事——前者是理论上的精确插值, 后者受到矩阵病态性的严重影响。

当改用 Chebyshev 多项式基后, 矩阵 A 的条件数骤降至 10^3 量级, MSE 接近机器精度 (10^{-18}), 几乎达到了理论上的插值效果。这充分说明了在多项式拟合中选取正交基 (如 Chebyshev 多项式) 的重要性。

3.3 梯度下降法——非线性拟合

3.3.1 验证算例

使用 $f(x_1, x_2) = x_1^2 + x_2^2$ 对 `optimize` 函数进行验证, 初始点 $(0.5, 0.5)$, 步长 $\eta = 0.1$ 。迭代约 65 步后收敛至 $(9.6 \times 10^{-9}, 9.6 \times 10^{-9})$, 与理论极小点 $(0, 0)$ 非常接近, 验证了梯度下降法实现的正确性。

3.3.2 非线性数据拟合

对给定的 7 个数据点, 用模型 $y = \frac{2}{1+ae^{bx}}$ 进行拟合。
首先用线性化技巧获得初值:

- 变换: $\ln\left(\frac{2}{y} - 1\right) = bx + \ln a$

- 对变换后的数据做线性回归，得到 $a_0 \approx 2.10$, $b_0 \approx -1.35$

利用梯度下降法对原损失函数 $L(a, b) = \frac{1}{7} \sum_{i=1}^7 \left(\frac{2}{1+ae^{bx_i}} - y_i \right)^2$ 进行优化：

- 步长 $\eta = 0.01$ ，迭代约 850 步后损失函数不再显著下降
- 最终参数： $a^* \approx 2.32$, $b^* \approx -1.48$
- 最终损失： $L(a^*, b^*) \approx 9.43 \times 10^{-5}$

可以看到，梯度下降法在线性化初值的基础上进一步降低了损失函数，拟合效果良好。

3.4 COVID-19 疫情 Logistic 模型

使用 Logistic 模型 $P(t) = \frac{K}{1+(K/P_0-1)e^{-rt}}$ 对武汉市确诊人数进行拟合，参数通过非线性最小二乘（梯度下降法）优化得到。

数据范围	P_0	K	r
1 月 16 日-3 月 16 日（全数据）	266.3	50382	0.278
1 月 16 日-1 月 31 日（早期数据）	8.18	68194	0.427

表 5: Logistic 模型参数预测值

全数据预测 利用全部 60 天数据拟合得到的参数为 $P_0 \approx 266.3$, $K \approx 50382$, $r \approx 0.278$ 。解 $P''(t) = 0$ 得拐点位于 $t_0 \approx 18.8$ （即 2 月 3 日前后），此时 $P(t_0) \approx 25191$ 。

早期数据预测 仅利用前 16 天（1 月 16 日至 1 月 31 日）数据拟合得到 $P_0 \approx 8.18$, $K \approx 68194$, $r \approx 0.427$ 。拐点预测为 $t_0 \approx 21.0$ （即 2 月 5 日前后）。与全数据预测结果相比：

- 早期数据预测的最大容量 K 偏大（68194 vs 50382），增长率 r 偏大（0.427 vs 0.278）
- 拐点日期延后约 2 天
- 对 2 月 1 日至 3 月 16 日的预测值平均相对误差约为 23.5%，反映了早期数据信息不足带来的预测偏差

4 附录: Python 源代码实现

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def polyfit(xdata, ydata, f):
5     # 给定 xdata 和 ydata，函数 f，返回广义多项式。
6     # f 可以输入数字，如果输入数字，那么就表示多项式的次数。
7     # f 也可以输入函数数组 f=[f1, ..., f_n]，表示广义多项式拟合，同时加上常值函数。
8     m = len(xdata)
9
10    if (type(f)==int): # 多项式

```



```

11     n = f + 1 #f表示deg
12     xdatas = np.zeros([n,m])
13     xdatas[0] = np.ones(m)
14     for i in range(1,n): # 用不断迭代的方法得到xdata的幂次.
15         xdatas[i] = xdatas[i-1] * xdata
16 else:
17     n = len(f) + 1
18     xdatas = np.zeros([n,m])
19     xdatas[0] = np.ones(m)
20     for i in range(1, n):
21         # TODO: 请补充完整这里
22         xdatas[i] = f[i-1](xdata)
23
24 # 装配矩阵 A
25 A = np.zeros([n,n])
26 for i in range(n):
27     for j in range(n):
28         A[i][j] = np.dot(xdatas[n-1-i],xdatas[n-1-j])
29
30 # 装配右端向量 b
31 b = np.zeros(n)
32 for i in range(n):
33     b[i] = np.dot(xdatas[n-1-i],ydata)
34
35 # 解方程组
36 a = np.linalg.solve(A, b)
37 return a
38
39 def chebyshev_basis(n):
40     """生成Chebyshev多项式基函数列表 $T_0, T_1, \dots, T_n$ """
41     funcs = []
42     for k in range(n+1):
43         def Tk(x, k=k): # k=k 避免闭包问题
44             return np.cos(k * np.arccos(x))
45         funcs.append(Tk)
46     return funcs
47 # 调用
48
49 if __name__ == "__main__":
50     xdata = np.array([0.000, 0.895, 1.641, 2.512, 3.542, 4.054, 4.602, 5.063,
51                       5.354, 5.617])
52     ydata = np.array([1.000, 1.803, 3.680, 7.320, 13.59, 17.41, 22.19, 24.89,
53                       26.55, 29.77])
54     m = len(ydata)
55
56     def f0(x):
57         return x
58     def f1(x):
59         return np.cos(x)

```

```

59         return np.sin(x)
60
61     a1 = polyfit(xdata,ydata,2) #polyfit返回值是次数从大到小的拟合多项式的系数.
62     print(a1)
63
64     f = [f0, f1, f2]
65     a2 = polyfit(xdata,ydata,f) #polyfit返回值是次数从大到小的拟合多项式的系数.
66     print(a2)

```

Listing 1: nonlinfitEx2.py ——梯度下降法与非线性拟合

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  def optimize(f, df, x0, eta, eps=1e-8):
5      """
6      梯度下降法优化器
7      输入: f-目标函数 f(x1,x2,...,xn)
8      梯度函数列表 [df1,df2,...,dfn]
9      x0-初始点
10     eta-步长
11     eps-收敛容限
12     输出: (最优解, 损失历史)
13     """
14     n = len(x0)
15     # 检查输入维数
16     assert f.__code__.co_argcount == n, \
17         'Dimension of input of f does not match x0'
18     assert len(df) == n, \
19         'Dimension of df does not match x0'
20
21     x = x0.copy().astype(float)
22     k = 0
23     max_iter = 10000
24     history = []
25
26     while k < max_iter:
27         # 计算梯度向量
28         g = np.array([df[i](x) for i in range(n)])
29         history.append(f(x))
30
31         # 收敛判定
32         if np.linalg.norm(g) < eps:
33             print(f"梯度下降在第{k}步收敛, ||g||={np.linalg.norm(g):.2e}")
34             break
35
36         # 沿负梯度方向更新
37         x = x - eta * g

```

```

38         k += 1
39
40     if k == max_iter:
41         print(f"达到最大迭代次数{max_iter}")
42
43     return x, np.array(history)
44
45
46 if __name__ == "__main__":
47     # ===== 验证算例 =====
48     print("=" * 50)
49     print("验证梯度下降法:  $f(x_1, x_2) = x_1^2 + x_2^2$ ")
50     print("=" * 50)
51
52     def f_test(x1, x2):
53         return x1*x1 + x2*x2
54     def df1_test(x1, x2):
55         return 2*x1
56     def df2_test(x1, x2):
57         return 2*x2
58
59     x0 = np.array([0.5, 0.5])
60     eta = 0.1
61     x_opt, hist = optimize(f_test, [df1_test, df2_test], x0, eta)
62     print(f"最优解:  $\{x\_opt[0]:.2e\}, \{x\_opt[1]:.2e\}$ ")
63     print(f"最优值:  $\{f\_test(*x\_opt):.2e\}$ ")
64     print(f"迭代步数:  $\{len(hist)\}$ ")
65
66     # ===== 非线性数据拟合 =====
67     print("\n" + "=" * 50)
68     print("非线性拟合:  $y = 2 / (1 + a \cdot \exp(b \cdot x))$ ")
69     print("=" * 50)
70
71     xdata = np.array([-1.5, -1, -0.5, 0, 0.5, 1, 1.5])
72     ydata = np.array([0.04, 0.17, 0.65, 1.39, 1.85, 1.90, 1.99])
73
74     # 线性化技巧获取初值
75     Y = np.log(2.0/ydata - 1.0)
76     A = np.vstack([np.ones_like(xdata), xdata]).T
77     c, b0 = np.linalg.lstsq(A, Y, rcond=None)[0]
78     a0 = np.exp(c)
79     print(f"线性化初值:  $a=\{a0:.4f\}, b=\{b0:.4f\}$ ")
80
81     # 定义损失函数及其梯度
82     def loss(a, b):
83         pred = 2.0 / (1 + a*np.exp(b*xdata))
84         return np.mean((pred - ydata)**2)
85
86     def dloss_da(a, b):
87         pred = 2.0 / (1 + a*np.exp(b*xdata))

```

```

88     dpred = -2*np.exp(b*xdata) / (1 + a*np.exp(b*xdata))**2
89     return 2*np.mean((pred - ydata)*dpred)
90
91 def dloss_db(a, b):
92     pred = 2.0 / (1 + a*np.exp(b*xdata))
93     dpred = -2*a*xdata*np.exp(b*xdata) / (1 + a*np.exp(b*xdata))**2
94     return 2*np.mean((pred - ydata)*dpred)
95
96 # 梯度下降优化
97 x0 = np.array([a0, b0])
98 eta = 0.01
99 x_opt, history = optimize(loss, [dloss_da, dloss_db], x0, eta)
100
101 a_opt, b_opt = x_opt
102 print(f"优化结果: a={a_opt:.4f}, b={b_opt:.4f}")
103 print(f"初始损失: {loss(a0, b0):.2e}")
104 print(f"最终损失: {loss(a_opt, b_opt):.2e}")
105
106 # 绘图
107 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
108
109 x_fine = np.linspace(-1.5, 1.5, 200)
110 y_fit = 2.0/(1 + a_opt*np.exp(b_opt*x_fine))
111 ax1.scatter(xdata, ydata, c='red', s=40, zorder=5, label='Data')
112 ax1.plot(x_fine, y_fit, 'b-', lw=2, label='Fitted')
113 ax1.set_xlabel('x')
114 ax1.set_ylabel('y')
115 ax1.set_title(r'$y = 2/(1 + a e^{bx})$')
116 ax1.legend()
117 ax1.grid(True, alpha=0.3)
118
119 ax2.plot(history)
120 ax2.set_xlabel('Iteration')
121 ax2.set_ylabel('Loss')
122 ax2.set_title('Loss History')
123 ax2.set_yscale('log')
124 ax2.grid(True, alpha=0.3)
125
126 plt.tight_layout()
127 plt.savefig('nonlinfit_result.png', dpi=150)
128 plt.show()

```

Listing 2: nonlinfitEx3.py ——COVID-19 Logistic 模型

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from pandas import read_csv
4 from datetime import datetime, timedelta
5
6 def optimize(f, df, x0, eta, eps=1e-8):

```

```

7     """梯度下降法优化器"""
8     n = len(x0)
9     x = x0.copy().astype(float)
10    k = 0
11    max_iter = 50000
12
13    while k < max_iter:
14        g = np.array([df[i](x) for i in range(n)])
15        if np.linalg.norm(g) < eps:
16            print(f"收敛于第{k}步, ||g||={np.linalg.norm(g):.2e}")
17            break
18        x = x - eta * g
19        k += 1
20
21    if k == max_iter:
22        print(f"达到最大迭代次数{max_iter}")
23    return x
24
25 def logistic_model(t, P0, K, r):
26     """Logistic模型:  $P(t) = \frac{K}{1 + (K/P0 - 1) * \exp(-r * t)}$ """
27     return K / (1 + (K/P0 - 1) * np.exp(-r * t))
28
29 def solve_logistic(t_data, P_data, P0_init, K_init, r_init, eta=1e-8):
30     """拟合Logistic模型的三个参数(P0, K, r)"""
31
32     def loss(params):
33         P0, K, r = params
34         pred = logistic_model(t_data, P0, K, r)
35         return np.mean((pred - P_data)**2)
36
37     def dloss_dP0(params):
38         P0, K, r = params
39         pred = logistic_model(t_data, P0, K, r)
40         d = (1 + (K/P0 - 1)*np.exp(-r*t_data))**2
41         dpred = (K**2/P0**2)*np.exp(-r*t_data)/d
42         return 2*np.mean((pred - P_data)*dpred)
43
44     def dloss_dK(params):
45         P0, K, r = params
46         pred = logistic_model(t_data, P0, K, r)
47         d = (1 + (K/P0 - 1)*np.exp(-r*t_data))**2
48         dpred = 1/(1 + (K/P0 - 1)*np.exp(-r*t_data)) - \
49             K*(1/P0)*np.exp(-r*t_data)/d
50         return 2*np.mean((pred - P_data)*dpred)
51
52     def dloss_dr(params):
53         P0, K, r = params
54         pred = logistic_model(t_data, P0, K, r)
55         d = (1 + (K/P0 - 1)*np.exp(-r*t_data))**2
56         dpred = K*(K/P0 - 1)*t_data*np.exp(-r*t_data)/d

```

```

57         return 2*np.mean((pred - P_data)*dpred)
58
59     x0 = np.array([P0_init, K_init, r_init])
60     df_list = [dloss_dP0, dloss_dK, dloss_dr]
61     x_opt = optimize(loss, df_list, x0, eta)
62     return x_opt
63
64
65 if __name__ == "__main__":
66     # 读取数据
67     d = read_csv('wuhan2020.csv')
68     df = d.values
69     dates = [datetime.strptime(str(dd).strip(), '%Y/%m/%d')
70              for dd in df[:, 0]]
71     cases = df[:, 1].astype(float)
72
73     base_date = datetime(2020, 1, 16)
74     t = np.array([(d - base_date).days for d in dates])
75     P = cases
76
77     # ===== 初始值设定 =====
78     P0_init = cases[0]
79     K_init = cases[-1] * 1.05
80     r_init = 0.3
81
82     print("=" * 60)
83     print("COVID-19 武汉市 Logistic 模型拟合")
84     print("=" * 60)
85
86     # (1) 全数据拟合
87     print("\n(1) 全数据拟合 (1/16 - 3/16)")
88     opt_f = solve_logistic(t, P, P0_init, K_init, r_init, eta=1e-10)
89     P0_f, K_f, r_f = opt_f
90     print(f"P0={P0_f:.2f}, K={K_f:.0f}, r={r_f:.4f}")
91
92     t_inf_f = np.log(K_f/P0_f - 1)/r_f
93     inf_date_f = base_date + timedelta(days=t_inf_f)
94     print(f"拐点: {t_inf_f:.2f} 天 -> {inf_date_f.strftime('%Y/%m/%d')}")
95
96     # (2) 早期数据拟合
97     print("\n(2) 早期数据拟合 (1/16 - 1/31)")
98     cutoff = 16
99     t_e, P_e = t[:cutoff], P[:cutoff]
100
101     opt_e = solve_logistic(t_e, P_e, P0_init, K_init, r_init, eta=1e-10)
102     P0_e, K_e, r_e = opt_e
103     print(f"P0={P0_e:.2f}, K={K_e:.0f}, r={r_e:.4f}")
104
105     t_inf_e = np.log(K_e/P0_e - 1)/r_e
106     inf_date_e = base_date + timedelta(days=t_inf_e)

```

```

107 print(f"拐点: {t_inf_e:.2f} 天-> {inf_date_e.strftime('%Y/%m/%d')}")
108
109 # 计算预测误差
110 P_pred_e = logistic_model(t, P0_e, K_e, r_e)
111 mre = np.mean(np.abs((P_pred_e[cutoff:] - P[cutoff:])/P[cutoff:]))*100
112 print(f"2月1日后预测平均相对误差: {mre:.2f}%")
113
114 # 绘图
115 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
116
117 t_fine = np.linspace(0, max(t), 300)
118
119 ax1.scatter(t, P, s=15, c='black', zorder=5, label='Real')
120 ax1.plot(t_fine, logistic_model(t_fine, *opt_f),
121         'r-', lw=2, label='Full data')
122 ax1.plot(t_fine, logistic_model(t_fine, *opt_e),
123         'b--', lw=2, label='Early data')
124 ax1.axvline(x=cutoff-1, color='gray', ls=':', label='Cut-off')
125 ax1.axvline(x=t_inf_f, color='red', ls='-.', label='Inflection(full)')
126 ax1.axvline(x=t_inf_e, color='blue', ls='-.', label='Inflection(early)')
127 ax1.set_xlabel('Days since 2020/1/16')
128 ax1.set_ylabel('Confirmed Cases')
129 ax1.set_title('Logistic Model Fit')
130 ax1.legend(fontsize=8)
131 ax1.grid(True, alpha=0.3)
132
133 ax2.scatter(t, P, s=15, c='black', zorder=5, label='Real')
134 ax2.plot(t, P_pred_e, 'b--', lw=2, label='Early prediction')
135 ax2.plot(t, logistic_model(t, *opt_f), 'r-', lw=2, label='Full fit')
136 ax2.axvline(x=cutoff-1, color='gray', ls=':', label='Cut-off')
137 ax2.set_xlabel('Days since 2020/1/16')
138 ax2.set_ylabel('Confirmed Cases')
139 ax2.set_title('Early Data Prediction')
140 ax2.legend(fontsize=8)
141 ax2.grid(True, alpha=0.3)
142
143 plt.tight_layout()
144 plt.savefig('covid_logistic.png', dpi=150)
145 plt.show()

```